

RoomScout

几何数据与数学模型说明

源码对应版 · 2026-09-08

本文把场景的数据结构、几何验证算法和修复选择机制对应到当前项目源码，说明每个模块接收什么、计算什么、提供什么结果。数学式采用可直接对照代码的记号。

系统工作方式

场景 JSON → 类型校验 → 几何诊断 → 模型提出移动候选
→ 每个候选独立复查 → 选择改善方案 → 下一轮或结束

几何校验器负责判定规则是否满足；大模型读取场景和非通过诊断，提出 MOVE 候选。当前模型请求只有文本 JSON，没有场景图片。规则策略也可以直接使用诊断内的 suggestions。

阅读导航

- 第 2 页：代码目录与数据模型
- 第 3 页：OBB、四元数、AABB 与轴对齐
- 第 4 页：SAT 碰撞与交叠体积
- 第 5 页：地面、接触、支撑面积与重心
- 第 6 页：支撑链、报告和参数
- 第 7 页：修复搜索、案例、接口与限制

项目目录与版本

/Users/initial/Library/Mobile Documents/com~apple~CloudDocs/room

本文中所有相对目录均以该路径为根目录。行号按生成时源码定位，后续编辑可能改变。

分支对应仓库：github.com/initial3366-droid/room1

源码提交：b65f587

适用边界：实心箱体、米制单位、统一世界坐标系、Z 轴向上。几何 pass 不等于真实物理稳定，不等于满足所有布局任务要求。

01 目录索引与数据模型

`app/contracts/models.py`

基础合同：Box、CameraPose、VerificationResult；Pydantic 校验尺寸、四元数及数据格式。

`app/verification/models.py`

GeometryObject、SceneSnapshot、Diagnostic、DiagnosisReport、MovePrescription、RepairResult。

`app/environment/geometry.py`

底层向量和旋转工具：dot、cross、rotate、axes、corners。

`app/verification/geometry.py`

几何规则主体：bounds、aligned、penetration、DeterministicGeometryVerifier。

`app/verification/repair.py`

候选应用、移动限制、全场景复查及逐轮选择。

`app/verification/llm.py`

LLMRepairPolicy：把场景和诊断发送给模型，解析移动候选。

`app/verification/io.py`

读取 JSON；把简写 position_m/size_m 转换为完整 geometry 结构。

`app/verification/port.py`

verify_layout：工件、版本和任务范围校验，应用布局后调用诊断。

`app/api/geometry.py`

HTTP 入口 /geometry/repair；调用修复流程并保存事件。

`configs/geometry-verifier-v1.json`

规则参数；Demo 场景见同目录 geometry-diagnosis-demo.json。

核心数据流

GeometryObject：object_id、geometry、movable、anchored、requires_support、support_id、可选局部重心。SceneSnapshot：对象集合、frame_id、floor_z_m、scene_id。

Diagnostic：规则、状态、涉及对象、原因、测量值、严重程度、可编辑变量和建议。DiagnosisReport：场景修订标识、验证器版本、诊断集合和总体状态。

MovePrescription：object_id + delta_m + diagnosis_id。RepairResult：最终场景、报告、接受的动作、迭代计数和结束状态。

输入合同拒绝重复对象 ID、自支撑、非法尺寸及坐标系不一致；support_id 缺省允许保留为 None，由诊断返回 unknown。

02 箱体、旋转与边界

代码位置：app/contracts/models.py:36 · Box

代码位置：app/verification/geometry.py:27 · bounds

OBB：有方向的三维箱体

中心 $c = (cx, cy, cz)$ ，尺寸 $s = (sx, sy, sz)$ ，半尺寸 $h = s/2$ 。旋转矩阵 R 的三列 u_1 、 u_2 、 u_3 是局部单位轴。

$p = c + \alpha_1 h_1 u_1 + \alpha_2 h_2 u_2 + \alpha_3 h_3 u_3$
 $-1 \leq \alpha_i \leq 1$ ；八个角点对应 $\alpha_i \in \{-1, +1\}$

position_m 表示中心，size_m 表示完整边长。一个边长 1 米、中心 (0,0,0.5) 的无旋转箱体，其 Z 范围为 [0,1]。

四元数与旋转

orientation_xyzw 存储 $q=(x,y,z,w)$ ，合同要求其模长平方接近 1。对局部向量 v ，旋转后得到 Rv ；局部重心也通过相同旋转变换到世界坐标。

$x^2 + y^2 + z^2 + w^2 \approx 1$
 $R = [1-2(y^2+z^2), 2(xy-zw), 2(xz+yw);$
 $2(xy+zw), 1-2(x^2+z^2), 2(yz-xw);$
 $2(xz-yw), 2(yz+xw), 1-2(x^2+y^2)]$

AABB：世界坐标轴上的范围

$lj = \min(k=1\dots 8) pk,j$
 $uj = \max(k=1\dots 8) pk,j, j \in \{x,y,z\}$

bounds() 返回 (low, high)。旋转箱体的 AABB 会包含额外空间，不能直接用其重叠代替精确 OBB 碰撞检测。最低 Z 坐标用于地面检测。

代码位置：app/verification/geometry.py:44 · aligned

对每个局部轴 u_i ： $\max(|u_i,x|, |u_i,y|, |u_i,z|) \geq 1 - \epsilon_{axis}$

三轴均满足时视为轴对齐。90° 轴置换也可满足；普通 45° 旋转不满足。当前支撑规则只在轴对齐条件下给出确定结果。

03 SAT 碰撞检测

代码位置：app/verification/geometry.py:61 · penetration

功能与候选分离轴

分离轴定理：如果存在一个方向，使两个凸盒体的投影分离，那么盒体不穿透。代码测试 A 的 3 个轴、B 的 3 个轴，以及两组轴两两叉乘产生的 9 个方向，最多 15 个。长度小于 $1e-10$ 的方向跳过，其余归一化为 n 。

候选方向： $\{u_i\} \cup \{v_j\} \cup \{u_i \times v_j\}$

$r_A(n) = \sum_{i=1 \dots 3} h_{A,i} |n \cdot u_i|$

$r_B(n) = \sum_{i=1 \dots 3} h_{B,i} |n \cdot v_i|$

$d = (c_B - c_A) \cdot n$

$p = r_A(n) + r_B(n) - |d|$

r_A 、 r_B 是投影半径， d 是中心在轴上的有符号距离。任意轴 $p \leq \epsilon_{\text{penetration}}$ 时返回 None；所有轴均超过容差才判定穿透。这里 p 表示沿该轴达到分离所需的移动量，在包含情形下不应混同于投影交集长度。

分离移动候选

$d \geq 0$ ： $\Delta c_A = -p n$

$d < 0$ ： $\Delta c_A = +p n$

移动 B 的对应候选： $\Delta c_B = -\Delta c_A$

返回按深度排序的 (depth, delta) 列表。建议只对当前物体对有效；必须复查它是否让对象撞到其他物体，或破坏原有支撑。

例子：两把椅子重叠

两个无旋转 1 米盒体的中心 X 坐标为 0 和 0.8，投影半径均为 0.5。 $p = 0.5 + 0.5 - 0.8 = 0.2$ 米。向左移 A 0.2 米，或向右移 B 0.2 米，可使这一对盒体刚好接触。

轴对齐盒体交叠体积

$\ell_j = \max(0, \min(u_{A,j}, u_{B,j}) - \max(l_{A,j}, l_{B,j}))$

$V = \ell_x \ell_y \ell_z$

仅双方均轴对齐时报告精确盒体交叠体积。一般旋转盒体体积为 None，SAT 仍能提供碰撞判断。全部物体对逐一检查， N 个物体有 $N(N-1)/2$ 对，主要复杂度为 $O(N^2)$ 。

04 地面与支撑几何

代码位置：app/verification/geometry.py:118 · diagnose

地面穿透

```
dfloor = zfloor - lz
dfloor > εpenetration → fail
建议位移 Δc = (0,0,dfloor)
```

物体最低点 -0.1 米、地面 0 米，则向上移动 0.1 米。物体悬空仍可通过地面穿透项，需要另做支撑检查。

垂直接触与水平重叠

支撑顶面 z_s 为地面高度，或声明支撑盒体的最高 Z 。以下 L 、 U 是物体与支撑物在 XY 上的交集边界。

```
g = lz - zs
Lj = max(lobject,j, lsupport,j)
Uj = min(uobject,j, usupport,j), j ∈ {x,y}
A = max(0, Ux-Lx) × max(0, Uy-Ly)
contact = (|g| ≤ εcontact) 且 (A > 0)
```

g 为正表示悬空，为负表示进入支撑物。接触不仅要求高度接近，也要求水平方向有重叠。地面按无限平面处理，其投影面积取物体底面范围。

重心与支撑边距

```
m = c + R mlocal ; 未提供局部重心时 mlocal = 0
M = min(mx-Lx, Ux-mx, my-Ly, Uy-my)
支撑有效 = contact 且 M ≥ Mmin
```

$M < 0$ 表示重心投影越界， $M = 0$ 表示位于边缘。地面支撑不计算有限边距；输入未给重心时假设均匀实心箱体。此判定是几何必要条件，不包含外力或动力学。

修复建议与边界

```
Δz = -g
水平不足时：Δx = (lsupport,x + usupport,x)/2 - mx
Δy = (lsupport,y + usupport,y)/2 - my
```

水平修正将重心移向支撑箱体中心，不保证最短。缺少 `support_id` 返回 `unknown`；指定的支撑对象不存在返回 `fail`；非轴对齐支撑返回 `unknown`。`anchored` 或 `requires_support=false` 的对象跳过局部支撑项。

05 支撑链、报告与参数

代码位置：app/verification/geometry.py:223 · grounded

支撑关系是有向图

$G = (V, E)$; $A \rightarrow B$ 表示 A 由 B 支撑

花瓶 $\rightarrow$ 桌子 $\rightarrow$ floor : 逐层确认局部支撑通过

递归到 floor 或声明的锚定对象返回 pass。节点不存在、循环或缺少可验证局部状态返回 unknown ; 遇到局部失败则传播其状态。seen 集合避免循环递归。代码只为局部支撑通过的对象追加 support_chain 诊断。

例如上层箱子接触桌面，但桌子悬空：箱子的局部 support 可以 pass，沿桌子向下追溯的支撑链会 fail。锚定是输入声明，不自动验证真实锚固结构。

代码位置：app/verification/geometry.py:137 · add

报告与严重程度

$\text{severity} = \min(1, \max(0, \text{amount}) / \text{severity_scale_m})$

总状态：存在 fail $\rightarrow$ fail ; 否则存在 unknown $\rightarrow$ unknown ; 否则 pass

add() 统一生成 diagnosis_id、measurements、editable_variables 和 suggestions。碰撞以穿透深度作为 amount，地面以穿地深度作为 amount，支撑以间隙或边距不足量计算。严重程度是排序指标，不是事故概率。

当前默认配置

目录：configs/geometry-verifier-v1.json

penetration_tolerance_m = 0.000001 m : 小于或等于容差的穿透按接触处理。

contact_tolerance_m = 0.001 m : 垂直间隙允许误差。

minimum_support_margin_m = 0.001 m : 重心离接触边缘的最小距离。

axis_tolerance = 1e-9 : 轴对齐判定的无量纲容差。

severity_scale_m = 0.1 m : 严重程度达到 1 的归一化尺度。

maximum_move_m = 2.0 m : 每次候选移动距离上限，不是总行程预算。

maximum_iterations = 20 : 默认循环上限；HTTP 请求默认 5，可指定 1 至 20。

load_config() 读取并校验配置；验证器 version 含配置指纹，scene.revision 含场景内容指纹，便于追溯和防止旧诊断作用于新场景。

06 修复搜索与使用说明

代码位置：app/verification/repair.py:41 · repair_scene

代码位置：app/verification/repair.py:17 · apply_move

受约束的逐轮局部搜索

$J(S) = (\text{失败项数}, \text{未知项数}, \text{失败项严重程度之和})$

接受条件： $J(S_{\text{new}}) \leq_{\text{lex}} J(S_{\text{old}})$

且 $\text{BadIDs}(S_{\text{new}}) \subseteq \text{BadIDs}(S_{\text{old}})$

位移限制： $\|\Delta c\|_2 \leq \text{maximum_move_m}$

按字典序先减少失败数量，再减少未知数量，最后比较失败严重程度。候选必须针对当前失败诊断的可编辑对象，固定物体不能移动；只更新位置，保留尺寸与旋转。每个候选独立应用于同一个当前场景，再全场景复查。

多个合格候选先比较 J ，再比较移动长度，最后用移动 JSON 文本打破平局。每轮接受一个候选。全部规则通过返回 pass；无改善候选返回 blocked；耗尽轮数仍未通过返回 iteration_limit。它不保证全局最优，也不会尝试先变差再变好的路径。

模型如何知道哪里出错

代码位置：app/verification/llm.py:68 · propose

模型上下文包含 scene、非 pass 的 diagnostics、最大移动量和候选数量。诊断已给出错误对象、原因、测量值及规则建议；模型返回 moves。最终几何是否通过由 diagnose() 决定。

Demo 对应结果

configs/geometry-diagnosis-demo.json：两把椅子重叠 0.2 米；floating_box 底面距地面 0.75 米；vase 的重心超出 pedestal 支撑区域约 0.1 米。相应规则建议包括椅子横移 0.2 米、箱子下降 0.75 米、花瓶向底座中心横移 0.6 米。具体接受顺序由候选和复查结果决定。

接口与复现入口

app/api/geometry.py：POST /geometry/repair 接收 scene 与 max_iterations。app/verification/port.py：verify_layout 校验工件哈希、版本、任务范围，应用候选布局后诊断；额外任务约束仍报告 unknown。

tests/test_geometry_verifier.py 覆盖碰撞、包含、支撑、修复和回放；tests/test_llm_repair.py 检查模型协议与移动门禁。现成调用脚本为 scripts/demo_llm_repair.py。

结论的准确范围

pass 仅表示当前盒体规则通过。系统未计算摩擦、受力、质量分布演化、变形和移动路径碰撞；未识别真实图像，也未评价美观、人体工学或任意任务语义。文档描述当前源码实现，不构成真实设备的执行许可。